Python, Behave and Mockito-Python

Here's an article that explores behaviour-driven development with Python for software developers interested in automated testing. Behaviour-driven development approaches the designing and writing of software using executable specifications that can be written by developers, business analysts and quality assurance together, to create a common language that helps different people communicate with each other.

This article has been written using Python 2.6.1, but any sufficiently new version will do.

Behave is a tool written by Benno Rice and Richard Jones, and it allows users to write executable specifications. Behave has been released under a BSD licence and can be downloaded from *http://pypi.python.org/pypi/behave*. This article assumes that version 1.2.2 is being used.

Mockito-Python is a framework for creating test doubles for Python. It has been released under an MIT licence, and can be downloaded from: *https://bitbucket.org/szczepiq/mockito-python/*. This article assumes the use of version 0.5.0, but almost any version will do.

Our project

For our project, we are going to write a simple program that simulates an automated breakfast machine. The machine knows how to prepare different types of breakfast, and has a safety mechanism to stop it when something goes wrong. Our customer has sent us the following requirements for it:

"The automated breakfast machine needs to know how to make breakfast. It can boil eggs, both hard and soft. It can fry eggs and bacon. It also knows how to toast bread to two different specifications (light and dark). The breakfast machine also knows how to squeeze juice from oranges. If something goes wrong, the machine will stop and announce an error. You don't have to write a user interface; we will create that. Just make easy-to-understand commands and queries that can be used to control the machine."

Note: Example code in this article is available as a zip archive at *https://github.com/tuturto/breakfast/zipball/master*.

Layout of the project

Let's start by creating a directory structure for our project:

```
mkdir -p breakfast/features/steps
```

The *breakfast* directory will contain our code for the breakfast machine. The *features* directory is used for storing specifications for features that the customer listed. The *steps* directory is where we specify how different steps in the specifications are done. If this sounds confusing, don't worry, it will soon become clear.

The first specification

In the case of Behave, the specification is a file containing structured natural language text that specifies how a given feature should behave. A feature is an aspect of a program—for example, boiling eggs. Each feature can have one or more scenarios that can be thought of as use cases for the given feature. In our example, the scenario is hard-boiling a single egg.

First, we need to tackle boiling the eggs. That sounds easy enough. Let's start by creating a new file in the features directory, called *boiling_eggs.feature*, and enter the following specification: